

**Course: Fundamentals of Generative AI and Large Language
Models: Theory and Practice**

**Week 2: Autoencoders: The Representation
Learning Model**

Today's Session Overview

- Regularization Techniques in AE: Sparse Autoencoders

NPTREL

Introduction

If the $\text{dim}(h) < \text{dim}(x)$, the model is forced to compress the input data. Hence, it learns the most important features.

If the $\text{dim}(h) > \text{dim}(x)$, the model may learn a trivial identity mapping, copying the input to the output without extracting meaningful features. This can lead to poor generalization or overfitting.

However, even if the hidden layer has many neurons, we can still make the model learn useful features by applying **sparsity** to the hidden activations.

In a Sparse Autoencoder (SAE), most hidden neurons stay inactive for most inputs, and only a few neurons become active to capture the important features of the input.

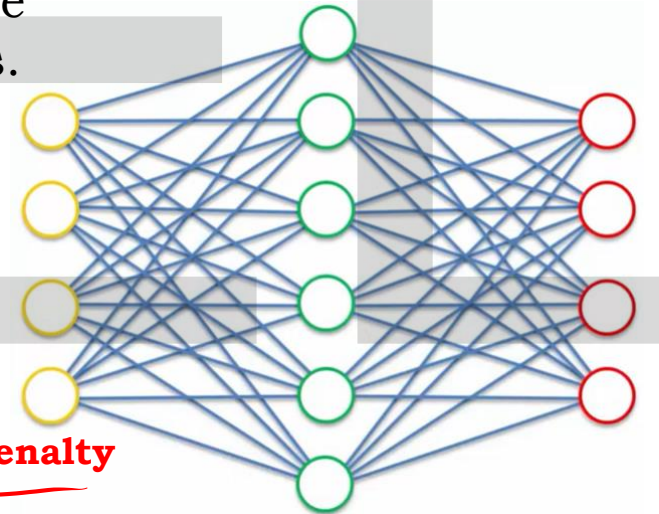
sparsity is imposed by adding a **sparsity penalty term** to the loss.

Total loss:

$$L = \underbrace{L_{\text{reconstruction}}}_{\text{how well input is reconstructed}} + \underbrace{\beta L_{\text{sparsity}}}_{\text{KL-divergence sparsity penalty}} \quad \left\{ \begin{array}{l} \text{L1 regularization on activations} \end{array} \right.$$

where:

- $L_{\text{reconstruction}}$: how well input is reconstructed
- L_{sparsity} : penalty when too many hidden neurons become active
- β : controls importance of sparsity



If one sample x is given to the network, then the activation of hidden neuron j is written as:

$$a_j^{(2)}(x)$$

This means:

Output of hidden neuron j in layer 2 for the input x

For the i -th training sample:

If the input is specifically the i -th sample, then it becomes:

$$a_j^{(2)}(x^{(i)})$$

This means:

Output of hidden neuron j when sample $x^{(i)}$ is passed through the network

Average over the whole training set:

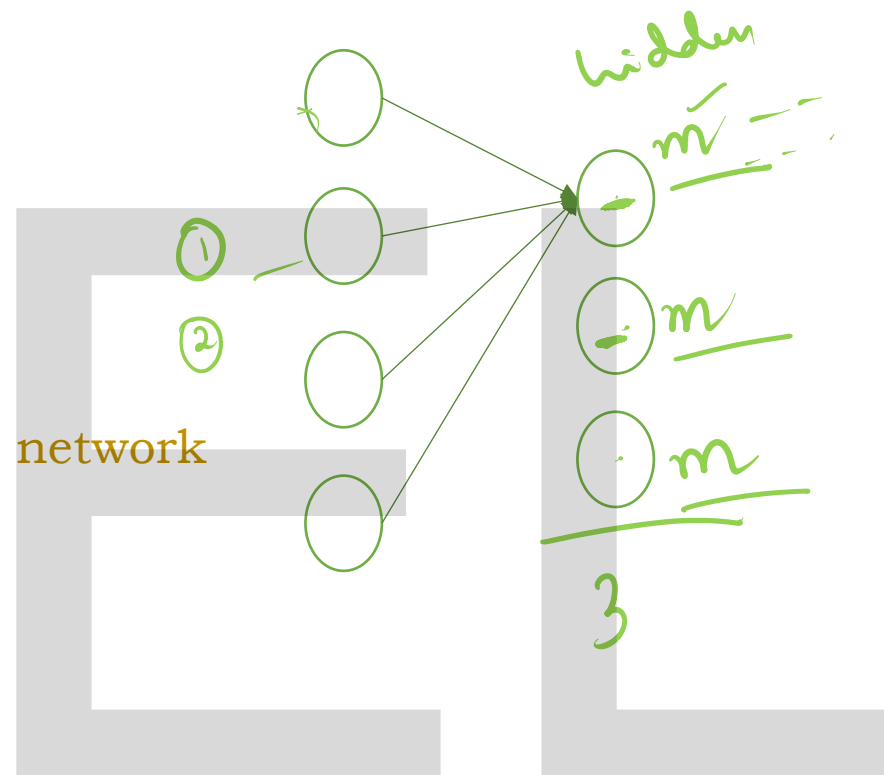
Now if there are m training samples, then average activation of hidden neuron j is:

$$\hat{\rho}_j = \frac{1}{m} \sum_{i=1}^m a_j^{(2)}(x^{(i)})$$

This means:

Take hidden neuron j , observe its activation for every training sample, and then compute the average

Within that hidden layer, only a small fraction of neurons should be active at a time, or on average each neuron should be active only rarely. The average activation of a neuron is close to 0.



KL Divergence as Sparsity Penalty

- ρ = desired average activation
- $\hat{\rho}_j$ = actual average activation of hidden neuron j

We want:

$$\hat{\rho}_j = \rho$$

SAE tries to enforce the constraint that on average the activation of any neuron in the hidden layer should be equal to ρ

- If $\hat{\rho}_j$ is close to ρ , penalty is small
- If $\hat{\rho}_j$ is far from ρ , penalty becomes large

This “how much penalty to give” is measured by:

$$KL(\rho \parallel \hat{\rho}_j)$$

KL divergence formula

$$KL(\rho \parallel \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$$

Important property

- If $\hat{\rho}_j = \rho$, then $KL(\rho \parallel \hat{\rho}_j) = 0$
- If $\hat{\rho}_j$ becomes much larger than ρ , KL divergence increases and a large penalty.

Total sparsity penalty

$$\sum_{j=1}^n KL(\rho \parallel \hat{\rho}_j)$$

This is added to the original reconstruction loss.

$$KL(\rho \parallel \hat{\rho}_j) = \underbrace{\rho \log \frac{\rho}{\hat{\rho}_j}}_{\text{active part}} + \underbrace{(1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}}_{\text{inactive part}}$$

A neuron's behavior has two sides: **active and inactive**
 If we check only the active side, we do not fully capture the neuron's behavior.
 So the KL formula uses: one term for active behavior and one term for inactive behavior and adds them together.
 That total mismatch becomes the **penalty**.

Let actual average activation also be:

$$\hat{\rho}_j = 0.05$$

So:

$$\text{actual active part} = \hat{\rho}_j = 0.05$$

$$\text{actual inactive part} = 1 - \hat{\rho}_j = 1 - 0.05 = 0.95$$

Now substitute:

$$\begin{aligned} KL(0.05 \parallel 0.05) &= 0.05 \log \frac{0.05}{0.05} + 0.95 \log \frac{0.95}{0.95} \\ &= 0.05 \log 1 + 0.95 \log 1 \end{aligned}$$

Since

$$\log 1 = 0$$

we get:

$$KL(0.05 \parallel 0.05) = 0$$

how often the neuron **should be active** vs
 how often the neuron is **actually active**

Meaning

➤ first term checks active part: 0.05 vs 0.05 → perfect

➤ second term checks inactive part: 0.95 vs 0.95 → perfect

So total penalty is zero.

how often the neuron **should be inactive** vs
 how often the neuron is **actually inactive**

ρ = desired average activation
 $\hat{\rho}_j$ = actual average activation

We take:

$$\rho = 0.05$$

This is desired average activation.

So:

➤ desired active part = 0.05

➤ desired inactive part
 = $1 - 0.05 = 0.95$

Sparse Autoencoders

Sample	h_1	h_2	h_3	h_4	h_5	h_6	h_7
$x^{(1)}$	0.001	0.30	0.004	0.002	0.006	0.40	0.003
$x^{(2)}$	0.002	0.25	0.003	0.001	0.005	0.35	0.004
$x^{(3)}$	0.004	0.28	0.006	0.003	0.007	0.45	0.002
$x^{(4)}$	0.003	0.32	0.005	0.002	0.008	0.38	0.003
$x^{(5)}$	0.005	0.27	0.004	0.001	0.006	0.42	0.005

For each neuron,

$$\hat{\rho}_j = \frac{1}{m} \sum_{i=1}^m a_j^{(2)}(x^{(i)})$$

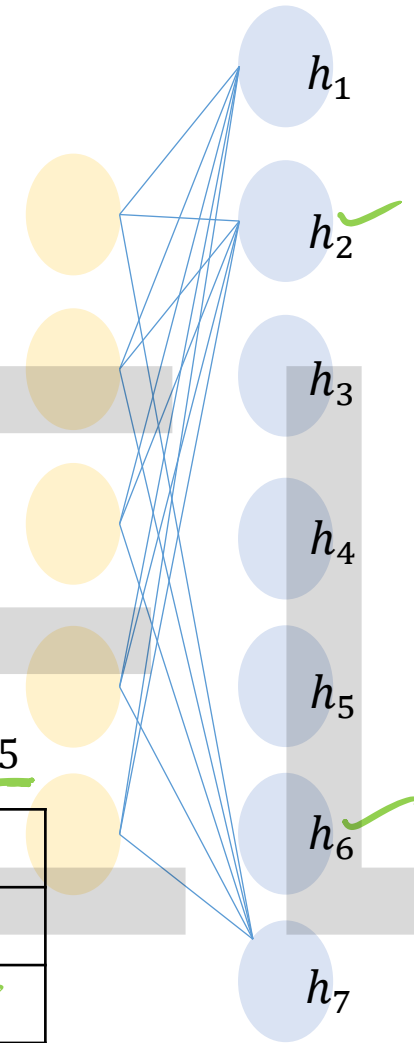
We compute **column-wise averages**.

$$\hat{\rho}_1 = \frac{0.001 + 0.002 + 0.004 + 0.003 + 0.005}{5} = 0.003$$

This is the **average activation of hidden neuron 1**.

Desired sparsity $\rho = 0.005$

h_1	$\hat{\rho}_1$	0.003	Close ✓
h_2	$\hat{\rho}_2$	0.284	Very far ✓
h_3	$\hat{\rho}_3$	0.0044	Very close ✓
h_4	$\hat{\rho}_4$	0.0018	Somewhat close
h_5	$\hat{\rho}_5$	0.0064	Close
h_6	$\hat{\rho}_6$	0.40	Very far
h_7	$\hat{\rho}_7$	0.0034	Close



So, neurons h_2 and h_6 are much too active.

They are firing too often. Sparse autoencoder does not want that.

Sparse Autoencoders

Desired sparsity $\rho = 0.005$

KL penalty (It uses the KL divergence between desired activation ρ and actual average activation $\hat{\rho}_j$):

h_1	$\hat{\rho}_1$	0.003 ✓
h_2	$\hat{\rho}_2$	0.284 ✓
h_3	$\hat{\rho}_3$	0.0044
h_4	$\hat{\rho}_4$	0.0018
h_5	$\hat{\rho}_5$	0.0064
h_6	$\hat{\rho}_6$	0.40
h_7	$\hat{\rho}_7$	0.0034

For neuron 1:

$$KL = (\rho \parallel \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$$

$$KL = (0.005 \parallel 0.003) = 0.005 \log \frac{0.005}{0.003} + (1 - 0.005) \log \frac{1 - 0.005}{1 - 0.003}$$

h_1	$KL(0.005 \parallel 0.003)$	0.0003
h_2	$KL(0.005 \parallel 0.284)$	0.13342 ✓
h_3	$KL(0.005 \parallel 0.0044)$	0.00001
h_4	$KL(0.005 \parallel 0.0018)$	0.00083
h_5	$KL(0.005 \parallel 0.0064)$	0.00007
h_6	$KL(0.005 \parallel 0.40)$	0.2091 ✓
h_7	$KL(0.005 \parallel 0.0034)$	0.00014

$$= 0.005 \times 0.221 + 0.995 \times (-0.0008)$$

$$= 0.00115 + (-0.000796)$$

$$= 0.000354 \checkmark$$

Clearly the main penalties come from: h_2 and h_6 and KL divergence gives a large penalty to h_2 and h_6

On average neuron is going to be inactive, that means whenever it is active it is really going to capture some relevant information.

Most neurons such as h_1, h_3, h_4, h_5, h_7 have very small penalties. But h_2 and h_6 have much larger penalties because they are firing too often. So after summation, the total penalty is dominated mainly by h_2 and h_6 .

Sparse Autoencoders

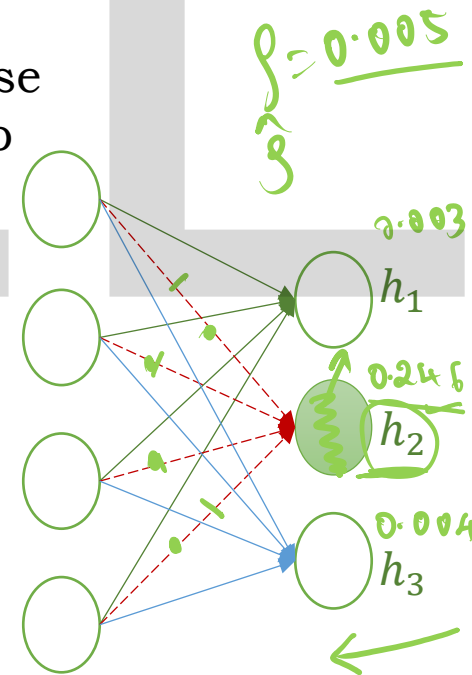
The sparse autoencoder objective is:

$$J_{\text{sparse}}(W, b) = L(x_i, \hat{x}_i) + \beta \sum_{j=1}^n KL(\rho \parallel \hat{\rho}_j)$$

Because the network is not allowed to use all hidden neurons freely, it can no longer reconstruct the input through a trivial dense representation. Instead, it is forced to use only a few active neurons efficiently, and this pressure encourages the hidden layer to learn meaningful features.

If a hidden neuron becomes too active, its KL divergence increases. This increases the sparsity penalty and therefore increases the total loss. During backpropagation, the network adjusts the weights and biases so that the activation of that neuron decreases.

In this way, overactive neurons are pushed to become less active and fire less often.



Next Session

➤ Contractive autoencoders

NPTREL

NIPTEEL

Thank you